

[Dashboard](#) / [My courses](#) / [2023S365114/246/247/248](#) / [Concluding Remarks](#) / [Exam](#)

Started on Monday, 26 June 2023, 1:45 PM

State Finished

Completed on Monday, 26 June 2023, 1:53 PM

Time taken 8 mins 5 secs

Grade 6.00 out of 10.00 (60%)

Question 1

Complete

Mark 1.00 out of 1.00

What is the best possible description for the output of the following snippet of code?

```
@torch.no_grad()
def compute_loss(network: nn.Module, loss_func: nn.Module, x: torch.Tensor, y: torch.Tensor):
    logits = network(x)
    err = loss_func(logits, y)
    err.backward()
    return err.item()
```

Select one:

- ☐ a. A Python `list` object without gradients
- ☐ b. A Python `list` object with gradients
- ☐ c. A `torch.Tensor` object with gradients
- ☒ d. A Python `float` object without gradients
- ☐ e. A `torch.Tensor` object without gradients

Question 2

Complete

Mark 1.00 out of 1.00

What is the most accurate description for the following snippet of code?

```
net = nn.Sequential()
net.add_module("features", FeatureNetwork(...))
net.add_module("classifier", nn.Linear(...))

state_dict = torch.load(...)
net.features.load_state_dict(state_dict)
nn.init.kaiming_uniform_(net.classifier.weight)
nn.init.zeros_(net.classifier.bias)

net.features.requires_grad_(True)
net.classifier.requires_grad_(True)
```

Select one:

- ☐ a. Preparation of a network with pre-trained features for transfer learning (without feature fine-tuning).
- ☒ b. Preparation of a network with pre-trained features for transfer learning with feature fine-tuning.
- ☐ c. Preparation of a fully pre-trained network for transfer learning (without feature fine-tuning).
- ☐ d. Preparation of a fully pre-trained network for transfer learning with feature fine-tuning.
- ☐ e. Preparation of a network that can not be used for transfer learning

Question 3

Complete

Mark 2.00 out of 2.00

Which two of the following concepts do **not** belong in the list of answer options? Concepts are said to belong together if they fit within one of the assignments you made this semester.

Select one or more:

- ☐ a. `model.decoder(model.encoder(x))`
- ☒ b. coupling layers
- ☐ c. reconstruction
- ☐ d. reparameterisation trick
- ☒ e. `model.features.requires_grad_(False)`

Question 4

Complete

Mark 0.00 out of 1.00

Which of the following lines of code implements the *projection* of projected gradient ascent, which is used to compute adversarial examples?

Select one:

- ☐ a. `logits = network(x + delta)`
- ☐ b. `err.backward()`
- ☒ c. `delta += 0.1 * delta.grad.sign()`
- ☐ d. `delta.clamp_(-0.1, 0.1)`
- ☐ e. `delta = torch.zeros_like(x, requires_grad=True)`

Question 5

Complete

Mark 1.00 out of 1.00

The code snippet below aims to produce the **objective to be minimised** for the generator of a GAN. You can assume the label for real (fake) labels to be one (zero). What is wrong with this code?

```
def generator_error(gan: nn.ModuleDict, x: torch.Tensor):  
    bce = nn.BCEWithLogitsLoss()  
    z = torch.randn_like(x)  
    with torch.enable_grad():  
        fakes = gan.discriminator(z)  
        logits = gan.generator(fakes)  
        fake_labels = torch.zeros_like(logits)  
        err = bce(logits, fake_labels)  
    return -err
```

Select one:

- ☐ a. The generator will be unable to receive weight updates.
- ☐ b. The generator will always produce the same outputs during training.
- ☐ c. The generator error is computed with the wrong kind of loss function.
- ☐ d. The generator weight updates will have the wrong sign.
- ☒ e. The generator and discriminator have been swapped.

Question 6

Complete

Mark 0.00 out of 1.00

How would you implement the log-determinant of an additive coupling layer?

☒ a.

```
def log_det(x):  
    return torch.sum(torch.log(self.conditioner(x[len(x) // 2:])))
```

☐ b.

```
def log_det(x):  
    return torch.ones_like(x)
```

☐ c.

```
def log_det(x):  
    return torch.zeros_like(x)
```

☐ d.

```
def log_det(x):  
    return -torch.sum(torch.log(self.conditioner(x[len(x) // 2:])))
```

☐ e.

```
def log_det(x):  
    return -torch.ones_like(x)
```

Question 7

Complete

Mark 1.00 out of 3.00

Match the code snippets to the corresponding ML research papers.

Hint: Try to find algorithms, figures or equations in the papers, there should be no need to read them.

Click on the selected ML research paper to view it:

- **NICE: Non-linear Independent Components Estimation**
- **Density Estimation using Real NVP**
- **Glow: Generative Flow with Invertible 1×1 Convolutions**

```
possible!
#         without flipping, identity mapping
always operates on same part of the input!
y = torch.cat([y2, y1], dim=1)
return y

def invert(self, y):
    y2, y1 = y.chunk(2, dim=1)
    log_scale, shift =
torch.chunk(self.conditioner(y2), 2, dim=1)
    x1, x2 = (y1 - shift) / log_scale.exp(), y2
    x = torch.cat([x1, x2], dim=1)
    return x

def log_det_jacobi(self, x):
```

NICE: Non-linear Independent Components Estimation

```
def __init__(self, num_channels: int):
    super().__init__()
    self.perm = nn.Conv2d(num_channels,
num_channels, 1, bias=False)

def forward(self, x):
    y = self._forward(x)
    ldj = self.log_det_jacobi(x)
    return y, ldj

def _forward(self, x):
    return self.perm(x)

def invert(self, y):
```

Glow: Generative Flow with Invertible 1×1 Convolutions

```
import torch
from torch import nn
```

```
class Coupling(nn.Module):
```

```
    def __init__(self, sub_net: nn.Module):
        super().__init__()
        self.conditioner = sub_net
```

```
    def forward(self, x):
        y = self._forward(x)
        ldi = self.log_det_jacobi(x)
```

Density Estimation using Real NVP

◀ Final Question

Jump to...